Unexpectedness is What You Need - Breaking The Filter Bubble

International Institute of Information Technology, Bangalore (IIIT-B)

***Abstract*—Recommender systems increasingly trap users in algorithmic "filter bubbles," optimizing for historical relevance at the cost of long-term user satisfaction and content discovery. Current industry standards for mitigating this rely heavily on rudimentary heuristics, equating true serendipity with random topical variance. This paper explores the classical 2014 paradigm of "unexpectedness" as a mathematical departure from a user's expected set, operating within a utility-maximizing framework. We critically analyze deep learning adaptations like PURS and UIRS-GNN, identifying core flaws such as the static embedding illusion, the loss of rating granularity, and the inability to natively process temporal decay. Finally, we propose a Non-Invasive Gated Attention architecture based on NOVA, which treats unexpectedness as a behavioral side-signal. By utilizing a dynamic gating fusor and multi-headed attention, this architecture natively balances serendipity and relevance based on real-time chronological context, offering a highly scalable solution to user fatigue.**

***Index Terms*—Recommender Systems, Filter Bubble, Unexpectedness, Serendipity, Transformers, Sequence Modeling**

I. Introduction and Motivation

In our daily digital experience, a frustratingly common pattern emerges: whether we are browsing for movies on platforms like Netflix or scrolling through short-form video feeds like YouTube Shorts, we inevitably hit a wall of boredom. Typically, a user discovers a genre they enjoy, consumes the highest-quality content within that niche, and is subsequently bombarded by the recommendation algorithm with highly similar, yet increasingly mediocre, variations of the same theme. This results in the familiar phenomenon of spending hours endlessly scrolling through options without ever committing to watching anything. Conversely, in short-form video feeds, we sometimes experience the opposite extreme—the algorithm suddenly injects content into the feed that is completely irrelevant or actively disliked by the user in a crude attempt to break the monotony.

This core problem is formally known as the "filter bubble." The filter bubble occurs because modern recommendation algorithms are hyper-optimized to exploit historical relevance. By continuously predicting what a user is most likely to click based strictly on their past behavior, the system traps the user in a tight, algorithmic silo of their own established interests. While this approach successfully maximizes short-term click-through rates, it is a significant problem because it leads to severe long-term user fatigue, a lack of content discovery, and ultimately, user churn.

Currently, the industry standard for mitigating the filter bubble relies heavily on rudimentary heuristics rather than true personalization. The most common approaches include:

1) **ϵ-Greedy Exploration:** The system randomly serves non-personalized or highly popular items a small percentage (ϵ) of the time to force exploration.
2) **Diversity-Aware Reranking:** Post-processing techniques, such as Maximal Marginal Relevance (MMR), are used to penalize items in a recommendation list that are too mathematically similar to one another, forcing topical variety.

The fatal flaw with these current industry standards is that they equate serendipity with mere randomness or topical variance. They fail to account for the chronological and psychological state of the user. Simply throwing a random, diverse item into a feed does not guarantee it will be a positive, unexpected discovery; more often than not, it is perceived as irrelevant noise. Recognizing this gap between random diversity and true serendipity, we set out to explore how to mathematically model and deliver satisfying "unexpectedness."

II. Literature Survey

A. Redefining the Problem: The 2014 Unexpectedness Paradigm

To build a theoretical foundation for solving the filter bubble, we started by analyzing the seminal 2014 paper, *On Unexpectedness in Recommender Systems: Or How to Better Expect the Unexpected* [1]. This paper fundamentally shifted our perspective by proving mathematically that true unexpectedness is not merely randomness; rather, it is a calculated deviation from a user's strict expectations that, when balanced with quality, maximizes their overall utility.

1) Formalizing Expectations and Unexpectedness: According to the paper, before we can recommend something unexpected, we must mathematically define what the user *does* expect. The authors define the "expected items" for a user u as a consideration set (E_u). This is a finite collection of typical items the user considers as candidates to fulfill their current needs, derived from their past interactions. Intuitively, any item that falls within this expected set possesses "zero unexpectedness".

The paper formally defines the unexpectedness of a candidate item i as its positive, unbounded distance from this expected set:

$$\delta_{u,i} = d(i; E_u) \tag{1}$$

However, the authors make a crucial psychological observation: unexpectedness is a unimodal function. There is a specific, ideal distance denoted as δ_u^*, which represents the user's "most preferred unexpected distance". If an item's distance is too close to the expected set, it is obvious and boring. If it departs too greatly (exceeding δ_u^*), the user will perceive it as completely irrelevant noise.

2) The Utility Metric Formulation: Traditional recommender systems typically formulate their objective simply as predicting the rating or relevance of an item. The 2014 paper departs from this by operating on the economic principle that users seek to maximize their overall "utility" (satisfaction).

They propose that the total utility of a recommendation is composed of two distinct parts: the utility derived from the item's quality, and the utility lost by deviating from the preferred level of unexpectedness.

First, the utility of quality is defined as:

$$U_{u,i}^q = q_u \times r_{u,i} + \epsilon_{u,i}^q \tag{2}$$

where $r_{u,i}$ is the item's rating, q_u is a positive constant reflecting how much that specific user values pure quality, and $\epsilon_{u,i}^q$ is a stochastic error term.

Second, the loss in utility caused by an item being too obvious or too bizarre is represented as:

$$U_{u,i}^\delta = -\lambda_u \times \phi(\delta_{u,i}; \delta_u^*) + \epsilon_{u,i}^\delta \tag{3}$$

where λ_u is a user-specific weight measuring their tolerance for redundancy, and the function ϕ captures how far the item's unexpectedness ($\delta_{u,i}$) has departed from the user's ideal sweet spot (δ_u^*).

Combining these components yields the final total utility equation that the system aims to maximize:

$$U_{u,i} = q_u \times r_{u,i} - \lambda_u \times \phi(\delta_{u,i}; \delta_u^*) + \epsilon_{u,i} \tag{4}$$

3) The Core Analogy: Relevance vs. Penalty: To summarize the brilliant intuition of this classical framework: traditional systems operate on the simple logic that $Score = Quality$. They purely chase relevance. The 2014 framework upgrades this logic to $Score = Quality - Penalty$.

The utility of a recommendation linearly increases with its rating. However, if the system suggests a highly rated item that is painfully obvious, or an item that is entirely disconnected from the user's interests, the $\lambda_u \times \phi$ term acts as a heavy mathematical penalty, dragging the total score down. By optimizing for this joint equation, a recommender system is forced to find items that are both high in quality and safely push the boundaries of the user's filter bubble without shattering it entirely.

B. Deep Learning Implementation: PURS and UIRS-GNN

To adapt this classical econometric framework into modern representation learning, we analyzed the Personalized Unexpected Recommender System (PURS) [2]. PURS successfully translated the 2014 utility equation into a deep learning architecture by predicting a hybrid utility score for user u and item i:

$$Utility_{u,i} = r_{u,i} + f(unexp_{u,i}) \times unexp_factor_{u,i} \tag{5}$$

where $r_{u,i}$ represents the estimated relevance (click-through-rate), $unexp_{u,i}$ is the mathematical unexpectedness distance, $f(\cdot)$ is a unimodal activation function, and $unexp_factor_{u,i}$ represents the user's personalized, temporal tolerance for unexpectedness. To maintain the unimodal nature of unexpectedness, they utilized the function $f(x) = x * e^{-x}$.

1) Embeddings and Relevance Calculation ($r_{u,i}$): The two models handle the generation of embeddings and relevance calculation differently:

The PURS Approach: PURS relies on explicit metadata. It utilizes an isolated Multi-Layer Perceptron (MLP) autoencoder to compress explicit user features (W_u) and item features (W_i) (like genres) into latent embeddings. To calculate relevance, it passes the user's history through a Self-Attentive Bidirectional GRU to capture the chronological mindset before feeding the final states into a predictive MLP.

The UIRS-GNN Approach: Recognizing that textual metadata is often unavailable, UIRS-GNN [3] relies purely on user and item IDs. It captures the user's temporal sequence using an Attention-based long short-term GRU (A-LSGRU). The A-LSGRU is specifically designed to assign different attention weights based on the target item. For example, if the candidate movie is Science Fiction, the attention mechanism dynamically assigns higher weights to the user's past Science Fiction interactions. It calculates a global preference vector and concatenates it with the user's most recent local preference, which is then fed into an MLP to output the final relevance score ($r_{u,i}$).

2) Unexpectedness ($unexp_{u,i}$) via Mean Shift Clustering: During our analysis, we asked a fundamental question: *Why can't we simply take the average of the user's past item representations, and calculate the distance of the new candidate movie from that average vector?*

This led us to the "Giant Blob" problem. Imagine a user who loves two completely different genres: high-octane Action movies and slow Romance movies. If we mathematically average the embeddings of those movies, the resulting vector lands dead in the center of the latent space—a giant, blurry blob that represents neither Action nor Romance.

PURS elegantly solved this using Mean Shift clustering. Instead of calculating a single naive average, Mean Shift iteratively locates areas of high data density to find distinct local maxima. It successfully identifies distinct "islands" of user interest (e.g., one distinct island for Action, another for Romance). The unexpectedness of a new candidate item w_* is then calculated as the weighted average distance between the new item and each of these historical islands (C_k):

$$unexp_{i_*,u} = \sum_{k=1}^{N} d(w_*, C_k) \times \frac{|C_k|}{\sum_{k=1}^{N} |C_k|} \tag{6}$$

To capture the user's current tolerance for this unexpectedness, the $unexp_factor_{u,i}$ was computed using a separate

Self-Attentive MLP applied strictly to a window of the user's most recent behaviors.

C. Pros of the PURS and UIRS-GNN Architectures

Overall, PURS effectively bridged the 2014 classical utility theory with modern deep learning. Furthermore, we investigated UIRS-GNN, an architectural upgrade that integrated Graph Neural Networks (GNNs) into this pipeline, introducing a major structural advantage:

1) **Enriching Sparse Data via Graph Hops (LightGCN):** In a standard model like PURS, if a user has only watched 3 items, their behavioral profile is incredibly sparse and weak. UIRS-GNN solves this by placing the user and items into a bipartite interaction graph. By using a streamlined graph model called LightGCN, the network performs "neighborhood aggregation". This means the model does not just look at those 3 items; it looks at the neighbors of those items, and the neighbors of those neighbors (multiple hops). Even with a tiny watch history, the user's representation becomes instantly rich and dense because it absorbs collaborative information from the wider network community before being passed to the sequence model.
2) **Solving Multi-Interest Blurring:** The application of Mean Shift clustering successfully captures multiple distinct user personas, validating the "island" approach over the naive "giant blob" average.

D. Identified Loopholes and Cons

Despite the robust theoretical translation, our critical analysis revealed three major bottlenecks that fundamentally limit these architectures:

1) **The Static Embedding Illusion:** While PURS and UIRS-GNN utilize recurrent networks (like A-LSGRU) to model the sequence of user behaviors, this is primarily used to understand the user's overall sequence state. The underlying item embeddings remain entirely static. Unexpectedness is highly dependent on the fluid representation of items in a specific context; by relying on static item vectors, the model fails to capture true chronological serendipity.
2) **Loss of Rating Intensity:** While baseline models like PURS and UIRS-GNN utilize ratings, they do so only to binarize the dataset, typically treating any rating > 3.5 as a positive interaction (1) and anything below as a negative (0). This mathematical reduction treats a 4-star "acceptable" movie and a 5-star "masterpiece" as identical signals. We identified that this ignores the magnitude of user feedback.
3) **Inability to Leverage Rich Side Information:** Because UIRS-GNN relies on LightGCN—which is explicitly designed for binary datasets with only user and item IDs—it structurally ignores highly valuable contextual data. It cannot natively incorporate exact timestamps, specific time gaps between interactions, or explicit rating values, all of which are critical for accurately modeling temporal boredom and stability.

III. Our Idea Evolution Journey

Our critical analysis of the existing literature led us to a fundamental realization: unexpectedness is highly determined by the user's immediate, chronological mindset. Relying on static item embeddings and rigidly defined, short-term windows (such as arbitrarily looking at the past 3 movies) is fundamentally insufficient to capture this.

To model a user's evolving context, we needed a dynamic architecture where the representation of an item actively shifts based on the sequence of items that preceded it. This naturally led us to pivot away from recurrent networks and adopt the Transformer architecture, specifically relying on its self-attention mechanism. However, determining exactly *how* to leverage Transformers to inject unexpectedness led to a multi-phase evolutionary process in our research.

A. Phase 1: Conceptualizing the Stability Metric

We started by attempting to mathematically model the user's psychological state: Are they bored, or are they actively exploring?

We hypothesized that if a user's long-term historical preferences closely match their immediate, short-term preferences, the user is in a highly "stable" state. In the context of recommendation, high stability means the user is stuck in a filter bubble, consuming the exact same type of content repeatedly. They are bored, and the system should provide a higher penalty for obvious items, delivering more unexpected recommendations. Conversely, if their recent preferences deviate significantly from their past preferences, stability is low. The user is actively exploring new niches, meaning the system should reduce the unexpectedness bonus and rely more heavily on pure relevance to satisfy their current exploratory intent.

To formalize this, we proposed a metric called *Preference Stability*, defined as the cosine similarity between a past preference embedding and a recent preference embedding.

B. Phase 2: Decoder-Only Windowing and Attention

To implement this stability metric dynamically, we utilized a decoder-only Transformer to process the user interaction sequence.

Let the user interaction sequence be defined as $S = [v_1, v_2, v_3, \ldots, v_t]$, where v_i represents the movie watched at step i. Each movie is mapped to a dense embedding vector $e_i \in \mathbb{R}^d$. To capture the chronological order of interactions, positional embeddings (p_i) are added to the item embeddings: $x_i = e_i + p_i$.

This sequence is processed by the decoder-only transformer to obtain contextualized hidden states $h_1, h_2, \ldots, h_t$. The final hidden state acts as the overarching session representation: $s = h_t$.

To estimate preference stability, we divided the sequence into two distinct parts:

- **Recent Interactions:** The last k items in the session representing immediate preferences, denoted as $Recent = [v_{t-k+1}, \ldots, v_t]$.
- **Past Interactions:** The remaining initial items representing historical baseline preferences, denoted as $Past = [v_1, \ldots, v_{t-k}]$.

Instead of treating all interactions equally, we applied an attention mechanism to compute the aggregated preference embeddings. The past preference embedding (p_{past}) and recent preference embedding (p_{recent}) are computed as weighted sums of their respective hidden states:

$$p_{past} = \sum_{i=1}^{t-k} \alpha_i h_i \quad \text{and} \quad p_{recent} = \sum_{i=t-k+1}^{t} \beta_i h_i \tag{7}$$

where α_i and β_i are learned attention weights.

With these two temporal vectors established, we calculate the Preference Stability using cosine similarity:

$$Stability = \cos(p_{past}, p_{recent}) \tag{8}$$

1) Integrating Stability into the Final Recommendation: Once stability is calculated, it acts as the dynamic weighting mechanism for the final recommendation score. First, the pure relevance score of a candidate item i is computed using the dot product between the session embedding and the candidate item embedding: $Rel(i) = s \cdot e_i$.

The unexpectedness of the candidate item ($Unexp(i)$) represents how different the item is from the user's current interests, and is evaluated using a customized distance function (e.g., $1 - \cos(s, e_i)$).

Finally, we proposed integrating relevance, unexpectedness, and our novel stability metric into a single ranking score:

$$Score(i) = Rel(i) + \lambda(1 - Stability)Unexp(i) \tag{9}$$

where λ is a hyperparameter controlling the overall influence of unexpectedness.

This equation perfectly captures our initial hypothesis: when stability is high (approaching 1), the term $(1 - Stability)$ approaches 0, reducing the weight of unexpectedness and forcing the model to break the bubble. Conversely, when stability is low, the influence of the unexpectedness term increases, allowing the system to recommend more diverse and novel items.

C. Phase 3: Deepening Transformer Understanding

To effectively implement our stability metric dynamically, we recognized the need to move beyond standard multi-layer perceptrons and recurrent networks. We undertook a deep dive into sequence modeling, specifically analyzing the seminal *Attention Is All You Need* architecture [4] and its recommendation-specific adaptation, *Self-Attentive Sequential Recommendation* (SASRec) [5].

Through this, we developed a rigorous understanding of three core mechanisms:

1) **Positional Encodings:** Injecting chronological order into an otherwise permutation-invariant attention mechanism.
2) **Multi-Headed Attention:** Allowing the model to jointly attend to information from different representation subspaces (e.g., one head tracking genre, another tracking release year).
3) **Masked Self-Attention:** Utilizing a lower-triangular matrix to prevent data leakage, ensuring that the prediction at time step t cannot "look ahead" at future interactions.

D. Phase 4: Customizing the Attention Mask for Stability

Our understanding of masked attention triggered a new architectural hypothesis. Standard SASRec utilizes a mask matrix M where $M_{ij} = -\infty$ if $j > i$, preventing the model from seeing future items [5].

We asked the following question: *Instead of calculating the past and recent embeddings as a post-processing step, why can't we manipulate this attention mask to natively calculate them within the Transformer layers?*

We proposed modifying the mask to strictly isolate the two contexts. Assuming a sequence of length t and a recent window of size k:

- **Recent Masking:** For an item in the recent window ($i > t - k$), we restrict its attention exclusively to other recent items.

$$M_{ij}^{(recent)} = \begin{cases} 0 & \text{if } t - k < j \leq i \\ -\infty & \text{otherwise} \end{cases}$$

- **Past Masking:** For an item in the historical past ($i \leq t - k$), we restrict its attention exclusively to other past items.

$$M_{ij}^{(past)} = \begin{cases} 0 & \text{if } j \leq i \leq t - k \\ -\infty & \text{otherwise} \end{cases}$$

By embedding these constraints directly into the self-attention formulation, the resulting hidden states would natively represent strictly isolated "past" and "recent" contexts. We hypothesized that this would lead to a much deeper, contextually aware calculation of the Preference Stability metric than our previous weighted-sum approach.

E. Remaining Loopholes in the Proposed Approach

While the custom-masked Transformer approach appeared theoretically sound, critical analysis revealed significant flaws in our conceptualization that we had not yet addressed:

1) **The Temporal Decay Factor (Time Gaps):** Our sequence model treated user interactions as evenly spaced discrete steps, entirely ignoring real-world timestamps. Time gaps profoundly affect psychological stability and boredom. For example, if a user watches ten action movies over three consecutive days, they are in a highly stable filter bubble and likely require unexpected recommendations. However, if those same ten interactions are spread out over three months, the natural time gap inherently resets their fatigue. We noted that a sufficiently large time gap can even make repetitive

content desirable again; for instance, a user might watch the exact same movie twice simply because the time gap between viewings was large enough to reset their interest. Current models fail to recognize that stability is a function of time, not just sequence order.

2) **The Heuristic Ceiling of Cosine Similarity:** Finally, we questioned the core mathematical definition of our stability metric. While calculating $\cos(p_{past}, p_{recent})$ is computationally efficient and elegant, human boredom is rarely a linear geometric distance. We concluded that a hardcoded cosine similarity heuristic might not be expressive enough to capture the highly complex, non-linear patterns of serendipity and fatigue that emerge across different user demographics.

IV. Proposed Final Architecture: Non-Invasive Gated Attention

To address the severe limitations regarding rating granularity and time-gap decay, we needed a method to inject this rich contextual data into the Transformer without destroying its mathematical foundation.

A. The Cross-Term Noise and Semantic Dominance Problems

Our initial instinct was to simply add this side information directly to the pure item embedding, identical to how positional encodings are injected: $x_i = e_i + p_i + r_i + t_i$ (where r_i is the rating embedding and t_i is the time gap embedding).

However, as highlighted in the TiSASRec [6] and NOVA [7] architectures, direct element-wise addition causes two fatal mathematical flaws:

1) **Cross-Term Noise:** When the self-attention layer calculates the dot product between two such composite vectors to find their correlation, the polynomial expansion creates meaningless, noisy cross-terms (e.g., mathematically multiplying an Item ID by a Time Gap).
2) **Semantic Dominance:** Side information can numerically overwhelm the pure item embedding. If a time gap vector is particularly large, it distorts the vector space, causing the network to group items by "when" they were watched rather than the semantic meaning of "what" the items actually are.

B. Unexpectedness as Behavioral Side Information

To bypass these issues, we drew heavy inspiration from the **NOVA (Non-inVasive self-Attention)** framework. In NOVA, side information is not just static metadata (like genre or year); it can be a fluid, chronological *behavioral pattern*.

This sparked our final breakthrough: the unexpectedness distance we are trying to capture ($f(cluster_distance)$) can simply be treated as another piece of behavioral side information, right alongside explicit ratings and time gaps.

To understand the power of this, we return to the classical 2014 utility function [1]:

$$Utility = q_u \times rating + \lambda_u \times unexpectedness \tag{10}$$

In our earlier Phase 1 approach, q_u was implicitly fixed, and λ_u was a manually hardcoded geometric heuristic defined by cosine similarity.

Our final proposal abandons these manual heuristics. Instead, we generate specific embeddings for every contextual signal and allow a neural **Gating Fusor** to independently calculate q_u and λ_u at every single chronological time step.

C. Embedding Generation and Gated Fusion

For a given sequence interaction, we first generate four distinct d-dimensional representations:

- **Pure Item (f_{item}):** Rather than a standard, isolated embedding matrix lookup, this is a structurally enriched embedding generated via a Graph Neural Network (GNN). By performing neighborhood aggregation across a user-item interaction graph (similar to LightGCN), this embedding natively captures high-order collaborative filtering signals. This ensures that even for users with sparse histories or items with cold-start issues, the baseline vector is already contextually rich before entering the Transformer.
- **Rating (f_{rating}):** The explicit user rating (e.g., 1 through 5) mapped to a trainable dense vector, preserving magnitude.
- **Time Gap (f_{time}):** The calculated time interval between the current and previous interaction, discretized into buckets and embedded.
- **Unexpectedness (f_{unexp}):** The offline-calculated distance from the user's Mean Shift clusters, projected via a linear layer into the d-dimensional space.

Instead of blindly adding them, we stack these features into a matrix F. The model calculates dynamic Gate weights (G) using a trainable weight matrix W^F and a sigmoid activation:

$$G = \sigma(FW^F) \tag{11}$$

The highly contextualized Integrated Representation (R) is then formed via a weighted linear combination:

$$R = \sum_i G^{(i)} f_i \tag{12}$$

This gating mechanism effectively acts as a dynamic, AI-driven utility function. To understand its power, consider the psychology of user boredom compared to the static 2014 econometric model, where the unexpectedness weight (λ_u) was a fixed parameter. Human fatigue is highly contextual. Imagine a user who binge-watches five highly similar action movies over a single weekend. Our neural gating mechanism processes these densely packed interactions, registering mathematically small time-gap embeddings (f_{time}) alongside highly clustered item representations. Recognizing this acute pattern of "filter-bubble fatigue," the trainable network autonomously spikes the sigmoid gate multiplier for the unexpectedness feature (f_{unexp}). In that specific chronological moment, the network assigns a massive priority to the unexpected factor—acting exactly like a temporary, high λ_u weight. Conversely, if that same user watches an action movie after a six-month hiatus, the large time-gap embedding signals a cognitive

reset. The network inherently understands the user is no longer bored, naturally closing the unexpectedness gate and falling back on high-relevance baseline recommendations. Thus, the Transformer continuously breathes and adapts to the user's real-time psychological state without relying on hardcoded rules.

D. Non-Invasive Multi-Headed Attention and the Core Analogy

Following the NOVA structure, we decouple the attention projection. We calculate the Query (Q) and Key (K) from the context-aware Integrated Representation (R), while the Value (V) is calculated strictly from the Pure Item Vector ($R^{(ID)}$):

$$Q = RW_Q, \quad K = RW_K, \quad V = R^{(ID)}W_V \tag{13}$$

To capture multiple behavioral patterns simultaneously, we employ **Multi-Headed Attention**. The projections are split into h distinct heads. One attention head might learn to strictly monitor time-gap boredom, while another head monitors pure rating quality. The attention for each head is calculated as:

$$head_i = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i \tag{14}$$

These heads are then concatenated and linearly projected to form the final hidden state.

The Core Analogy: This architecture can be understood through the "Librarian Metaphor." The Query (Q) and Key (K) act as the search system. Because they are built from R, they contain all the contextual knowledge (e.g., "Find me a 5-star movie, watched recently, that is highly unexpected"). However, the actual book pulled off the shelf—the Value (V)—contains only the pure text of the book ($R^{(ID)}$). This ensures the neural network retrieves highly serendipitous items without ever corrupting its internal semantic understanding of the catalog.

E. Industry Practicality, Advantages, and Complexities

Architectural Superiority and Potential: This architecture represents our most robust and comprehensive proposal because it resolves the fundamental flaws of all preceding iterations. First, it shatters the "static embedding illusion" seen in PURS and UIRS-GNN. By transforming the 2014 classical utility framework into an end-to-end differentiable gating network, the model no longer relies on hardcoded windows, manual cosine thresholds, or static heuristics. It dynamically evaluates the user's psychological fatigue at every single sequence step, autonomously deciding when to break the filter bubble. Second, the non-invasive attention mechanism allows us to natively inject crucial, multidimensional behavioral signals—exact rating granularity, precise time gaps, and the mathematical unexpectedness distance—without corrupting the pure item semantics with cross-term noise.

The Future Potential of the Gated Architecture: Beyond solving immediate technical hurdles, this proposal establishes a foundation for truly "human-centric" AI. By treating unexpectedness as a behavioral side-signal, we move away from the industry's obsession with pure accuracy and toward a model that understands the rhythm of human discovery. This architecture has the potential to solve the long-standing "stagnation" problem in platforms like YouTube or Netflix, where users feel the system "knows them too well" to ever surprise them. By giving the Transformer the agency to autonomously open the unexpectedness gate when it senses sequence-level boredom, we create a recommendation experience that feels alive, adaptive, and genuinely serendipitous.

Practical Industry Scalability: Beyond its theoretical elegance, the model offers a massive advantage in real-world production environments. Compared to the heavy online message-passing overhead of pure graph architectures like UIRS-GNN, our pipeline is highly efficient. The Mean Shift clustering and GNN-based base item representations can be computed asynchronously offline. Meanwhile, the real-time online inference relies strictly on the highly optimized Transformer, allowing for the scalable, low-latency recommendations required by modern industry platforms.

Training Complexity: However, training this end-to-end framework poses significant challenges. Because standard Binary Cross-Entropy (BCE) loss strictly optimizes for predicting the exact next interacted item, a "lazy" network might realize that recommending highly relevant, safe items yields an acceptable baseline accuracy. It may default to pushing the unexpectedness gate weight to near zero. Successful training requires careful hyperparameter tuning and ensuring the training dataset contains sufficient historical examples of positive unexpected discoveries to force the network to value the f_{unexp} embedding.

The Black Box Disadvantage: The primary theoretical drawback is a total loss of interpretability. In Phase 1, our cosine similarity metric provided a hard, mathematically transparent definition of stability. If the system recommended a novel item, we could look at the cosine angle and logically explain exactly why. By migrating to a dense Transformer gating mechanism, this interpretable logic is completely swallowed by the weights of a black box.

F. Future Work: Explainability via KG-GNN

To solve this black-box limitation in future iterations, we propose integrating Knowledge-Enhanced Graph Neural Networks (KG-GNNs), such as KGAT (Knowledge Graph Attention Network) [8].

By replacing our offline mathematical clustering algorithm with a Knowledge Graph-embedded GNN, we can generate highly structured, relation-aware unexpectedness embeddings. When the Transformer's gating mechanism triggers a serendipitous recommendation, the underlying KG-GNN can explicitly trace the relational path (e.g., $User \rightarrow$ watched $\rightarrow$ $MovieA \rightarrow$ shares secondary genre with $\rightarrow MovieB$). This will completely resolve the interpretability flaw while maintaining the state-of-the-art predictive accuracy of the non-invasive sequence model.

REFERENCES

[1] Adamopoulos, P., & Tuzhilin, A. (2014). *On Unexpectedness in Recommender Systems: Or How to Better Expect the Unexpected*. ACM Transactions on Intelligent Systems and Technology (TIST), 5(4), 1-32.

[2] Li, P., Que, M., Jiang, Z., Hu, Y., & Tuzhilin, A. (2020). *PURS: Personalized Unexpected Recommender System for Improving User Satisfaction*. Fourteenth ACM Conference on Recommender Systems (RecSys '20).

[3] Authors of UIRS-GNN (2023). *Unexpected interest recommendation based on sequence modeling and graph neural networks*. Complex & Intelligent Systems, 9, 3819–3833.

[4] Vaswani, A., et al. (2017). *Attention is all you need*. Advances in neural information processing systems, 30.

[5] Kang, W. C., & McAuley, J. (2018). *Self-attentive sequential recommendation*. ICDM.

[6] Li, J., Wang, Y., & McAuley, J. (2020). *Time Interval Aware Self-Attention for Sequential Recommendation (TiSASRec)*. WSDM '20.

[7] Liu, C., et al. (2021). *Non-invasive Self-attention for Side Information Fusion in Sequential Recommendation (NOVA)*. AAAI.

[8] Wang, X., He, X., Cao, Y., Liu, M., & Chua, T. S. (2019). *KGAT: Knowledge graph attention network for recommendation*. KDD '19.